



# TIP101 | Intro to Technical Interview Prep

Intro to Technical Interview Prep Summer 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)

Personal Member ID#: 151287

Need help? Post on our [class slack channel](#) or email us at [support@codepath.org](mailto:support@codepath.org)

Getting Started

Learning with AI



► Course Info

Course Progress

Unit 1

Unit 2

Unit 3

Unit 4

Unit 5

Unit 6

Unit 7

Unit 8

Overview

Session #1

Session #2

Assignment

Cheatsheet

Resources

Report

## Unit 5: Session 2

### Manipulating Linked Lists

In this session, students will delve into the intricate world of linked lists, focusing on both singly and doubly linked list structures. They will engage in a series of hands-on problems that challenge them to perform operations such as insertion, deletion, copying, and value manipulation within linked lists.

You can find all resources from today including the session deck, session recording, and more on the [resources tab](#)



#### Part 1 : Instructor Led Session

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.



#### Part 2: Breakout Session

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as “solving the problem” - please engage

warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
  - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

► **Note on Expectations**

## Problem Solving Approach

We will approach problems using the six steps in the UMPIRE approach.

**UMPIRE: Understand, Match, Plan, Implement, Review, Evaluate.**

We'll apply these six steps to the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem
- **Match** identifies common approaches you've seen/used before
- **Plan** a solution step-by-step, and
- **Implement** the solution
- **Review** your solution
- **Evaluate** your solution's time and space complexity and think critically about the advantages and disadvantages of your chosen approach.

## 💡 Unit 5 Cheatsheet

To jumpstart your journey into object-oriented programming and linked lists, we've put together a guide to common concepts and syntax you will use throughout Unit 5 breakout problems. Use this cheatsheet as a quick reference guide as you work through the problems below.

### Problem Set Version 1

#### ▼ Problem 1: Battle Pokemon

Given the `Pokemon` class below, copy the code and add it to your IDE.

```
class Pokemon():
    def __init__(self, name, hp, damage):
        self.name = name
        self.hp = hp # hit points
        self.damage = damage # The amount of damage this pokemon does to its opponent every att

    def attack(self, opponent):
        pass
```

Then, write a method `attack()` that takes in a `Pokemon` object `opponent` and decreases `opponent`'s `hp` by their self's `damage` amount.

If damaging the opponent leads to the opponent having an `hp` of 0 or less, the function should set the opponent's `hp` to 0 and print out `"<Opponent name> fainted"`.

Otherwise, the function should print out `"<Pokemon name> dealt <damage> damage to <opponent name>"`.

Example Usage:

```
pikachu = Pokemon("Pikachu", 35, 20)
bulbasaur = Pokemon("Bulbasaur", 45, 30)

opponent = bulbasaur
pikachu.attack(opponent)
```

Example Output: `Pikachu dealt 20 damage to Bulbasaur`

### ▼ ✨ AI Hint: Writing Methods

*Key Skill: Use AI to explain code concepts*

This problem requires you to write your own method! Try it yourself, but if you get stuck, you can:

- Check out the [Unit 5 Cheatsheet](#)
- Use an AI tool like ChatGPT or GitHub Copilot to show you examples of how to write methods in Python

Close Section

## ▼ Problem 2: Convert to Linked List

A **linked list** is a new data type that, similar to a normal list or array, allows us to store pieces of data sequentially. The difference between a linked list and a normal list lies in how each element is stored in a computer's memory.

In a normal list, individual elements of the list are stored in adjacent memory locations according to the order they appear in the list. If we know where the first element of the list is stored, it's really easy to find any other element in the list.

In a linked list, the individual elements called **nodes** are not stored in sequential memory locations. Each node may be stored in an unrelated memory location. To connect nodes together into a sequential list, each node stores a reference or pointer to the next node in the list.

Using the provided `Node` class below, create the normal Python list `["Jigglypuff", "Wigglytuff"]` as a linked list.

```
class Node:
    def __init__(self, value, next=None):
```

```
self.value = value
self.next = next
```

Example Usage (after completing the problem with variable names `node_1` and `node_2`):

```
print(node_1.value, "->", node_1.next.value)
print(node_2.value, "->", node_2.next)
```

Example Output:

```
Jigglypuff -> Wigglytuff
Wigglytuff -> None
```

Result Linked list: `Jigglypuff -> Wigglytuff`

### ▼ ✨ AI Hint: Linked Lists

*Key Skill: Use AI to explain code concepts*

This question requires you to be familiar with Linked Lists, a incredibly useful but sometimes tricky data structure. To help, we've included a review of linked lists [Unit 5 Cheatsheet](#)

You can also use an AI tool like ChatGPT or GitHub Copilot to get more examples or ask follow-up questions. You can use the following prompt as a starting point:

*"You're an expert computer science tutor. Can you help me understand linked lists conceptually, using analogies to real-world objects?"*

Once you understand the concept of Linked Lists, you can also ask follow-up questions like:

*"Can you provide examples of how to implement a linked list in Python, and explain how each part works?"*

*"Here is a provided Linked List class: (CODE). Can you give me an example of how to access the data in this linked list?"*

### ▼ Problem 3: Add First

Write a function `add_first()` that takes in a `head` of a linked list and a `new_node` from the `Node` class as parameters.

It should insert `new_node` as the new **head** of the `linked_list`. The function returns `new_node`.

*Note: The "head" of a linked list is the first element in the linked list. Equivalent to `Lst[0]` of a normal list.*

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

def add_first(head, new_node):
    pass
```

Example Usage:

```
# Using the Linked List from Problem 2
print(node_1.value, "->", node_1.next.value)

new_node = Node("Ditto")
node_1 = add_first(node_1, new_node)

print(node_1.value, "->", node_1.next.value)
```

Example Output:

```
Jigglypuff -> Wigglytuff
Ditto -> Jigglypuff
```

## ▼ Problem 4: Get Tail

Write a function `get_tail()` that takes in the `head` of a linked list as a parameter.

Return the value of the `tail`. If the list is empty, return `None`.

*Note: The "tail" of a list is the last element in the linked list. Equivalent to `lst[-1]` in a normal list.*

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

def get_tail(head):
    pass
```

Example Usage:

```
# Build: num1 -> num2 -> num3
num1 = Node("num1")
num2 = Node("num2")
num3 = Node("num3")
num1.next = num2
num2.next = num3

head = num1
tail = get_tail(head) # or get_tail(num1)
print(tail)           # expected: num3
```

Example Output: `num3`

### ▼ 💡 Hint: Linked List Traversal

This problem requires you to traverse a linked list. In other words, it requires you to iterate over the nodes of a linked list. For a break down of how to traverse a linked list, check out the [Unit 5 Cheatsheet](#).

## ▼ Problem 5: Replace Node

Using the `Node` class, write a function `ll_replace()` that updates in place every node whose `value == original` to have `value = replacement`. The function returns `None`.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

def ll_replace(head, original, replacement):
    pass

def to_string(head): # to test your list
    parts, cur = [], head
    while cur:
        parts.append(str(cur.value))
        cur = cur.next
    return " -> ".join(parts) if parts else "EMPTY"
```

Example Usage:

```
num3 = Node(5)
num2 = Node(6, num3)
num1 = Node(5, num2)
# initial linked list: 5 -> 6 -> 5

head = num1
ll_replace(head, 5, "banana")
# updated linked list: "banana" -> 6 -> "banana"
```



## ▼ Problem 6: List Nodes

Write a function `listify_first_n()` that takes in a `head` of a linked list and a non-negative integer `n` as parameters.

The function returns a list of values of the first `n` nodes.

- If `n` is greater than the length of the linked list, return a list of the values of all nodes in the linked list.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

def listify_first_n(head, n):
    pass
```

Example Usage:

```
# Linked List: a -> b -> c
head = a
lst = listify_first_n(head, 2)
print(lst)

# Linked List: j -> k -> l
head2 = j
lst2 = listify_first_n(head2, 5)
print(lst2)
```

Example Output:

```
['a', 'b']
['j', 'k', 'l']
```

Close Section

## ▼ Problem 7: Insert Value

Write a function `ll_insert()` that takes in a `head` of a linked list, a value `val`, and an index `i` as parameters.

The function should insert a new `Node` with value `val` at index `i` of the linked list, then return the head of the linked list.

- If `i` is greater than the length of the list, insert the new node at the end of the list.

*Hint: Linked lists do not have built-in indices so you will need to track the indices yourself.*

Write a helper function to help you print the new list!

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

def ll_insert(head, val, i):
    pass
```

Example Usage:

```
# Linked List: 3 -> 8 -> 12 -> 9
ll_insert(head, 20, 2)

# result Linked List: 3 -> 8 -> 20 -> 12 -> 9
```

Close Section

## ▼ Problem 8: Linked Listify

Write a function `list_to_linked_list()` that takes in a list `lst` as a parameter and converts it to a linked list. The function should return the **head** of the linked list.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
```

```
self.next = next
```

```
def list_to_linked_list(lst):  
    pass
```

Example Usage:

```
normal_list = ["Betty", "Veronica", "Archie", "Jughead"]  
linked_list = list_to_linked_list(normal_list)  
  
# This prints ONLY the head node's value:  
print(linked_list.value)    # => "Betty"  
  
# (Optional) Traverse to print the whole linked list:  
current = linked_list  
while current:  
    end_arrow = " -> " if current.next else "\n"  
    print(current.value, end=end_arrow)  
    current = current.next  
  
# Print the head node's VALUE:  
print(linked_list.value)    # expected: Betty
```

Example Output:

```
Betty
```

```
Betty -> Veronica -> Archie -> Jughead
```

Close Section

## ▼ Problem 9: Doubly Linked List

One of the drawbacks of a linked list is that it's difficult to go backwards, because each `Node` only knows about the `Node` in front of it. (E.g., `A -> B -> C` )

A **doubly linked list** solves this problem! Instead of just having a `next` attribute, a doubly linked list also has a `prev` attribute that points to the `Node` before it. (E.g., `A <-> B <-> C` )

Given the `Node` class for a doubly linked list below, recreate the list `["Poliwag", "Poliwhirl", "Poliwrath"]` as a doubly linked list.

```
class Node:
    def __init__(self, value, next = None, prev = None):
        self.value = value
        self.next = next
        self.prev = prev
```

Example Usage: (after completing the problem with variable names `poliwag`, `poliwhirl`, and `poliwrath`):

```
print(poliwhirl.prev.value, "<->", poliwhirl.value, "<->", poliwhirl.next.value)
```

Example Output:

```
Poliwag <-> Poliwhirl <-> Poliwrath
```

Close Section

## ▼ Problem 10: Print Backwards

Write a function `print_reverse()` that takes in the `tail` of a doubly linked list as a parameter.

It should print out the values of the linked list in reverse order, each separated by a space.

```
class Node:
    def __init__(self, value, next = None, prev = None):
        self.value = value
        self.next = next
        self.prev = prev
```

```
def print_reverse(tail):  
    pass
```

Example Usage: *(using the doubly linked list from Problem 9)*

```
#           (head)           (tail)  
# Linked List: Poliwag <-> Poliwhirl <-> Poliwrath  
print_reverse(poliwrath)
```

Example Output: Poliwrath Poliwhirl Poliwag

Close Section

## Problem Set Version 2

- ▶ Problem 1: Poker Two-Pair Hand
- ▶ Problem 2: Barbie Linked List
- ▶ Problem 3: Insert Value First
- ▶ Problem 4: Linked List Length
- ▶ Problem 5: Delete Tail
- ▶ Problem 6: Greatest Node
- ▶ Problem 7: Pop Node
- ▶ Problem 8: Find Middle Node
- ▶ Problem 9: Create Double Links
- ▶ Problem 10: Double to Single

## Problem Set Version 3

- ▶ Problem 1: Calculate Tournament Placement
- ▶ Problem 2: Update Linked List Sequence
- ▶ Problem 3: Insert Node as Second Element
- ▶ Problem 4: Increment Linked List Node Values

- ▶ Problem 5: Copy Linked List
- ▶ Problem 6: Find Minimum in Linked List
- ▶ Problem 7: Remove Node by Value from Linked List
- ▶ Problem 8: Move Tail to Front of Linked List
- ▶ Problem 9: Convert Singly Linked List to Doubly Linked List
- ▶ Problem 10: Find Length of Doubly Linked List from Any Node